

// JavaJub

Собес в Альфа-Банк

Alfa Digital

Java Middle

Вопросы, задачи и подготовка
к live-coding и техническому интервью

Стек

Java 11+ • Spring Boot • Kafka • PostgreSQL • Docker • Kubernetes • JUnit •
Microservices

[// содержание](#)

Содержание

01	Про Альфа-Банк и формат собеседа	3
02	Стек по вакансии	4
03	Java Core — что точно спросят	5
04	Коллекции	7
05	Многопоточность и JMM	8
06	Spring и Spring Boot	10
07	Hibernate и Spring Data JPA	12
08	SQL и PostgreSQL	13
09	Микросервисы и Kafka	14
10	Docker, Kubernetes, CI/CD	16
11	Тесты: JUnit, Mockito, TDD	17
12	Практические задачи (live-coding)	18
13	План подготовки + чек-лист	21

01 Про Альфа-Банк и формат собеседа

Альфа-Банк — крупнейший частный банк России, №1 в рейтинге работодателей hh.ru (2025). IT-подразделение Alfa Digital разрабатывает мобильный банк, антифрод-системы, платёжные сервисы, кредитные конвейеры и десятки других продуктов. Для Java Middle это означает: микросервисы, высокие нагрузки, промышленный стек.

По отзывам кандидатов на DreamJob, собеседование в Альфу длится 1–2 часа и включает live-coding, вопросы по SQL, обсуждение опыта и проектов. Интервьюеры ценят умение рассуждать вслух и объяснять свои решения.

► Как устроен процесс

Этап	Длительность	Что проверяют
Скрининг HR	20–30 мин	Мотивация, ЗП, стек, ожидания
Тех-интервью	60–90 мин	Java Core + live-coding + SQL + Spring
Системный дизайн	30–45 мин	Архитектура микросервисов, Kafka
Проверка СБ	до недели	Стандартная для банков (анкета, беседа)
Оффер	—	ДМС, стоматология, премии KPI, удалёнка

ФИШКА · Ключевая особенность

По отзывам кандидатов: в Альфе ценят умение рассуждать вслух и писать код в реальном времени. Это не экзамен по теории — это проверка инженерного мышления. Если не знаешь — скажи «не сталкивался, но предположу...» — честность ценят.

02 Стек по вакансии

На апрель 2026 года Альфа-Банк активно нанимает Java-разработчиков в Alfa Digital. Стек определён по вакансиям на hh.ru и job.alfabank.ru. Проекты: антифрод, кредитный конвейер, Альфа-Онлайн, электронное подписание.

► Обязательный минимум

- Java 11+ (по некоторым вакансиям до Java 21), опыт от 1–2 лет
- Spring / Spring Boot — знать, как работает «под капотом» (прокси, автоконфигурация)
- Микросервисная архитектура — понимание принципов, декомпозиция, взаимодействие
- Kafka — брокеры сообщений, топики, партиции, consumer groups
- PostgreSQL / Oracle — SQL, индексы, транзакции, EXPLAIN ANALYZE
- Docker, Kubernetes / OpenShift — контейнеризация и оркестрация
- JUnit 5, Testcontainers, WireMock — обязательно умение писать тесты
- CI/CD (Jenkins / GitLab CI) — понимание пайплайна
- Git, code review — ежедневный рабочий инструмент

► Будет плюсом

- MongoDB, Elasticsearch — NoSQL-хранилища
- Redis / Hazelcast / Infinispan — кэширование данных
- RxJava / Project Reactor — реактивное программирование
- Kotlin — встречается в некоторых командах
- Опыт в банковских / финтех проектах

ВНИМАНИЕ · Что это значит

Стек Альфы заточен под микросервисы и высокие нагрузки. Kafka, Kubernetes и кэширование — не «плюсы», а рабочие инструменты. На собеседовании спросят: «А что будет, если Kafka-consumer упадёт?» или «Как обеспечить идемпотентность?»

03 Java Core — что точно спросят

По отзывам кандидатов в банковские IT: equals/hashCode + HashMap — абсолютные чемпионы по частоте. Спрашивают все, абсолютно все. Далее идут Stream API, исключения и String Pool.

► Базовые вопросы

- Что такое JDK, JRE, JVM?

→ JVM — виртуальная машина, исполняет байт-код. JRE = JVM + стандартные библиотеки (нужно для запуска). JDK = JRE + инструменты разработки: javac, jdb, jar. С Java 9 JRE как отдельный дистрибутив не поставляется.

- Области памяти JVM.

→ Heap — хранение объектов, управляется GC. Stack — фрейм для каждого вызова метода (локальные переменные, ссылки, примитивы). Metaspace (до Java 8 — PermGen) — метаданные классов, пул интернированных строк. PC Register — адрес текущей инструкции потока. Native Method Stack — для JNI-вызовов.

- Почему String immutable?

→ Четыре причины: 1) безопасность — строки передаются в конструкторы файлов, БД, сетевых соединений, изменение привело бы к уязвимостям; 2) потокобезопасность — можно шарить между потоками без синхронизации; 3) кэширование hashCode — вычисляется один раз; 4) String Pool — экономия памяти.

- Контракт equals/hashCode.

→ Если `a.equals(b) == true`, то `a.hashCode() == b.hashCode()`. Обратное необязательно (коллизии допустимы). `equals(null) → false`. Переопределяешь один — переопределяй оба. Практический пример: положить объект в HashSet, изменить поле в hashCode — объект потеряется, `contains()` вернёт false.

- Что сломается, если hashCode() вернёт константу?

→ Все объекты попадут в один bucket HashMap. Java 7: связный список $O(n)$. Java 8+: при 8 коллизиях и `capacity ≥ 64` — перестройка в red-black tree $O(\log n)$. Это лучше $O(n)$, но деградация по сравнению с $O(1)$.

- checked vs unchecked исключения.

→ Checked: от Exception (не RuntimeException) — компилятор требует throws/catch. Примеры: IOException, SQLException. Unchecked: от RuntimeException и Error. Примеры: NullPointerException, IllegalArgumentException, ClassCastException. Современные фреймворки (Spring, Hibernate) предпочитают unchecked.

- try-with-resources.

→ Автоматически вызывает close() в finally. Ресурс реализует AutoCloseable. Закрывание в обратном порядке объявления. Если и try, и close бросают исключение — close-исключение подавляется (getSuppressed()). С Java 9 можно передать effectively final переменную.

- Функциональный интерфейс.

→ Ровно один абстрактный метод (SAM). @FunctionalInterface — опциональная аннотация, но защищает от случайного добавления второго метода. Основные: Function<T,R>, Predicate<T>, Consumer<T>, Supplier<T>, UnaryOperator<T>, BiFunction<T,U,R>, Comparator<T>, Runnable, Callable<V>.

- Stream API: промежуточные vs терминальные.

→ Промежуточные (filter, map, flatMap, sorted, distinct, peek, limit, skip) — ленивые, возвращают Stream, не выполняются пока нет терминальной. Терминальные (collect, forEach, count, reduce, findFirst, toList) — запускают конвейер. Стрим одноразовый — после терминальной повторно нельзя.

- **map vs flatMap.**
→ *map: T → R, один элемент → один. flatMap: T → Stream<R> с уплощением. Пример: List<List<String>> → flatMap(Collection::stream) → плоский Stream<String>. Для Optional: flatMap избегаем Optional<Optional<T>>.*
- **Optional — когда?**
→ *Для возвращаемых значений методов. НЕ для полей (не Serializable), НЕ для параметров, НЕ для коллекций (пустая лучше). Антипаттерны: Optional.get() без isPresent(), Optional в полях, Optional<List>.*
- **final — класс, метод, поле.**
→ *Класс — нельзя наследовать (String, Integer). Метод — нельзя переопределить. Поле — нельзя переприсвоить ссылку, но можно менять внутреннее состояние (final List — можно add/remove). effectively final — нужна для лямбд.*
- **Абстрактный класс vs интерфейс.**
→ *Абстрактный класс: состояние, конструкторы, реализация. Наследуется один. Интерфейс: контракт, любое число. Java 8 — default/static. Java 9 — private. Когда что: общее состояние → абстрактный класс, контракт для разных иерархий → интерфейс.*

► Задачи «Что выведет?»

В банках любят такие задачи — проверяют понимание, а не заученность. Вот типичные примеры из реальных собеседов:

Тест 1. Передача по значению

```
Integer i = Integer.valueOf(1);
inc(i);
System.out.println(i); // ?

private static void inc(Integer i) { i++; }
```

Ответ: 1. Java передаёт ссылки по значению. i++ создаёт новый объект Integer(2) и присваивает локальной переменной. Оригинал не меняется. То же самое со String — неизменяемый объект.

Тест 2. Integer cache

```
Integer i1 = Integer.valueOf(127);
Integer i2 = Integer.valueOf(127);
System.out.println(i1 == i2); // ?

Integer i3 = Integer.valueOf(128);
Integer i4 = Integer.valueOf(128);
System.out.println(i3 == i4); // ?
```

Ответ: true, false. IntegerCache кэширует значения от -128 до 127. Для 127 — один объект, ссылки совпадают. Для 128 — два разных объекта. Мораль: для объектов всегда equals(), никогда ==.

Тест 3. Stream API — ленивость

```
List<Integer> nums = List.of(1, 2, 3, 4, 5);
nums.stream()
    .map(x -> { System.out.print(x+" "); return x; })
    .filter(x -> x > 2)
    .map(x -> { System.out.print(x+" "); return x; })
    .toList();
```

Ответ: 1 2 3 3 4 4 5. Стрим обрабатывает элементы вертикально: каждый элемент проходит всю цепочку. Элементы 1 и 2 не проходят filter → печатаются один раз. Элементы 3,4,5 проходят → печатаются дважды.

СОВЕТ · Лайфхак

На вопрос про equals/hashCode приведи практический пример: «если положить объект в HashSet и изменить поле в hashCode — объект потеряется, contains() вернёт false». Это показывает понимание, а не заученность.

04 Коллекции

HashMap — абсолютный чемпион. По опыту кандидатов в банки: «обсасывают бедную мапу со всех сторон». Готовься рассказывать устройство, коллизии, treeify threshold, resize, null-ключи.

- Основные интерфейсы Collection Framework.
 - Collection (List, Set, Queue) и Map (отдельно, НЕ наследует Collection). List — упорядоченный с дубликатами. Set — без дубликатов. Queue — очередь. Map — пары ключ-значение. Реализации: ArrayList, LinkedList, HashSet, TreeSet, HashMap, TreeMap, LinkedHashMap, ConcurrentHashMap, ArrayDeque, PriorityQueue.
- Как устроен HashMap?
 - Массив бакетов Node<K,V>[]. Размер — степень двойки (default 16). Индекс: $(n-1) \& \text{hash}(\text{key})$, hash — XOR верхних 16 бит с нижними (spread). Коллизии — связный список. Java 8+: при TREEIFY_THRESHOLD=8 элементах в бакете И capacity ≥ MIN_TREEIFY_CAPACITY=64 — перестройка в red-black tree. При уменьшении до 6 — обратно (UNTREEIFY_THRESHOLD).
- load factor и resize.
 - Порог 0.75 по умолчанию. При $\text{size} \geq \text{capacity} * \text{loadFactor}$ — resize: массив вдвое + перехэширование ВСЕХ элементов (дорого!). Совет: если знаешь количество элементов — задай $\text{initialCapacity} = \text{expectedSize} / 0.75 + 1$.
- HashMap vs ConcurrentHashMap.
 - HashMap: не потокобезопасен, допускает 1 null-ключ, null-значения. ConcurrentHashMap: потокобезопасен. Java 7 — Segment. Java 8+ — CAS + synchronized на головах бакетов (лучше параллелизм). null-ключи и null-значения ЗАПРЕЩЕНЫ. computeIfAbsent — атомарная «проверь и вставь».
- ArrayList vs LinkedList.
 - ArrayList: $O(1)$ доступ по индексу, $O(n)$ вставка в середину, дружелюбно к CPU-кэшу (непрерывная память). LinkedList: $O(1)$ вставка в начало/конец (если есть ссылка), $O(n)$ доступ по индексу. На практике ArrayList почти всегда лучше — даже вставка в середину быстрее из-за локальности кэша.
- Fail-fast итератор.
 - ConcurrentModificationException при изменении коллекции не через сам итератор. Реализован через modCount. Не гарантирован в многопоточной среде — только best effort. Альтернатива: CopyOnWriteArrayList (fail-safe, работает с копией).
- TreeMap vs LinkedHashMap.
 - TreeMap: красно-чёрное дерево, $O(\log n)$, ключи отсортированы (Comparable/Comparator). LinkedHashMap: HashMap + двусвязный список — порядок вставки. accessOrder=true — LRU-кэш. removeEldestEntry() для ограничения размера.
- Как сделать List неизменяемым?
 - List.of(1,2,3) — Java 9+, immutable, null запрещён. List.copyOf(list) — Java 10+, глубокая неизменяемая копия. Collections.unmodifiableList(list) — обёртка (изменения в оригинале видны!). Предпочитай List.of или List.copyOf.

05 Многопоточность и JMM

Для Middle ждут уверенное понимание `synchronized`, `volatile`, `happens-before` и пулов потоков. JMM — обязательная тема для банков с высоконагруженными системами.

- `synchronized`.
 - Захватывает монитор объекта. `Instance-метод` — монитор `this`. `Static` — монитор `Class`. Блок — указанный объект. Только один поток. `Reentrant`: поток может повторно захватить свой монитор (счётчик). Гарантирует: *mutual exclusion + happens-before (видимость)*.
- `volatile`.
 - Видимость между потоками (запрет кэширования в регистрах/L1-кэше) + запрет *reordering* вокруг `volatile`. НЕ гарантирует атомарность `i++` (`read-increment-write = три операции`). Для атомарных — `AtomicInteger`.
- `happens-before`.
 - Ключевое отношение JMM. Если `A happens-before B`, то записи в `A` видны в `B`. Примеры: `unlock` → `lock` того же монитора. `volatile write` → `read`. `Thread.start()` → первая инструкция потока. Последняя инструкция → `Thread.join()`. `final`-поля видны после конструктора.
- Атомич-классы.
 - `AtomicInteger`, `AtomicLong`, `AtomicReference`. `CAS (Compare-And-Swap)` — атомарная инструкция CPU. `Lock-free`. Методы: `get`, `set`, `compareAndSet`, `incrementAndGet`. Для счётчиков с высоким contention — `LongAdder` (лучше масштабируется).
- `wait/notify` — почему `while`?
 - *Spurious wakeup*: JVM/ОС может разбудить поток без `notify`. `while (!condition) { wait(); }`. Важно: вызывать ТОЛЬКО внутри `synchronized` на том же объекте, иначе — `IllegalMonitorStateException`. `notify()` будит один поток, `notifyAll()` — все.
- Пулы потоков.
 - `newFixedThreadPool(n)` — фикс. число, `LinkedBlockingQueue`. `newCachedThreadPool` — неограниченно (ОПАСНО — ООМ). `newScheduledThreadPool` — периодические задачи. В проде: `ThreadPoolExecutor` вручную с `corePoolSize`, `maxPoolSize`, `BlockingQueue` и `RejectedExecutionHandler`.
- `ThreadLocal` — опасность.
 - С пулами потоков: поток переиспользуется, `ThreadLocal` остаётся от предыдущей задачи (утечка данных/памяти). Обязательно: `try { ... } finally { threadLocal.remove(); }`. В `Reactor/WebFlux` — не работает, нужен `Context`.
- `Deadlock`.
 - Два+ потока ждут друг друга. Условия Коффмана: *mutual exclusion, hold and wait, no preemption, circular wait*. Избегать: упорядочить захват мониторов (`lock A` → `lock B`), `tryLock` с таймаутом. Обнаружение: `jstack`, `VisualVM`.
- `CompletableFuture`.
 - `thenApply: T → R`. `thenCompose: T → CF<R> (flatMap)`. `thenCombine`: объединение двух CF. *exceptionally*: обработка ошибки. Аsync-варианты выполняются в `ForkJoinPool.commonPool()` или указанном `Executor`.

06 Spring и Spring Boot

В вакансии Альфы прямо указано: «знаешь, как работает Spring/Spring Boot под капотом». Поверхностного понимания недостаточно — будут копать в прокси, автоконфигурацию, жизненный цикл бинов.

- IoC и DI.
 - IoC — не ты создаёшь зависимости, а контейнер. DI — реализация: зависимости внедряются через конструктор/сеттер/поле. Преимущества: слабая связанность, подмена реализаций, тестируемость (можно подставить мок).
- Какой DI лучше?
 - *Constructor injection*. Поля можно *final*, явно видны зависимости, легко тестировать без контекста (*new Service(mockRepo)*), невозможен циклический DI при старте (сигнал о проблеме в дизайне). С Spring 4.3: если один конструктор — *@Autowired* не нужен.
- Жизненный цикл бина.
 - Инстанцирование → DI → Aware-интерфейсы → *BPP.postProcessBefore* → *@PostConstruct* → *InitializingBean* → *init-method* → *BPP.postProcessAfter* → ГОТОВ → *@PreDestroy* → *DisposableBean* → *destroy-method*. *BeanPostProcessor* — точка для создания прокси (*@Transactional*, *@Async*).
- *@Transactional* — под капотом.
 - Spring создаёт прокси-обёртку. *JDK dynamic proxy* (если интерфейс) или *CGLIB* (наследование). Прокси: открывает транзакцию (*PlatformTransactionManager*), вызывает метод, *commit* при успехе, *rollback* при *RuntimeException/Error*. *Checked HE* откатывают — нужно *rollbackFor*.
- *self-invocation*.
 - *this.method()* минует прокси → *@Transactional/@Async/@Cacheable* не работают. Решения: вынести метод в другой бин (рекомендуемый), инжектировать *self* через *@Lazy* или *ObjectProvider*, использовать *AspectJ compile-time weaving*.
- *Propagation*.
 - *REQUIRED* (default) — присоединяется/создаёт. *REQUIRED_NEW* — всегда новая, приостанавливает текущую (аудит/логирование). *NESTED* — *savepoint* внутри текущей. *SUPPORTS* — присоединяется если есть, нет — без. *MANDATORY* — требует существующей. *NOT_SUPPORTED* — приостанавливает. *NEVER* — если есть → исключение.
- *@SpringBootApplication*.
 - *@Configuration* + *@EnableAutoConfiguration* + *@ComponentScan*. Автоконфигурация через *@Conditional*: если *DataSource* в *classpath* — настроим JPA. Список в *META-INF/spring/...AutoConfiguration.imports* (Spring Boot 3+, раньше *spring.factories*).
- *Scopes* бина.
 - *singleton* (default — один на контекст), *prototype* (новый при каждом запросе), *request* (один на HTTP-запрос), *session*, *application*, *websocket*. Важно: инжект *prototype* в *singleton* через *Provider<T>* или *ObjectFactory<T>*, иначе *prototype*-бин создастся один раз.
- Обработка исключений.
 - *@RestControllerAdvice* + *@ExceptionHandler*. *ErrorDto* с *code*, *message*, *timestamp*. HTTP-статусы: 400 (валидация), 404 (не найдено), 409 (бизнес-конфликт), 500. *MethodArgumentNotValidException* для *@Valid*, *ConstraintViolationException* для *@Validated* на *query*-параметрах.

07 Hibernate и Spring Data JPA

- Что такое ORM?

→ *Object-Relational Mapping* — маппинг Java-объектов на таблицы. *Hibernate* — реализация JPA. Позволяет работать с БД через объекты. НО: всегда следи за тем, какой SQL реально выполняется (*Hibernate* может генерировать неоптимальные запросы — включи `hibernate.show_sql`).

- Состояния сущности.

→ *Transient* — `new User()`, *Hibernate* не знает. *Persistent/Managed* — связан с сессией, *dirty checking* автоматически синхронизирует изменения. *Detached* — сессия закрыта/`detach`. *Removed* — помечен на удаление. *persist*: *Transient*→*Managed*. *merge*: *Detached*→*Managed*. *remove*: *Managed*→*Removed*.

- *LazyInitializationException*.

→ Обращение к *lazy*-полю после закрытия сессии. Решения (от лучшего к худшему): DTO-проекция (`SELECT NEW`), `JOIN FETCH (@Query)`, `@EntityGraph(attributePaths)`, `@BatchSize(size=100)`. НЕ решение: `OpenSessionInView` (анти-паттерн — создаёт N+1 в представлении).

- N+1 — детально.

→ 1 `findAll()` + N доп. запросов на *lazy*-коллекции. 1000 заказов → 1001 SQL. Обнаружение: `hibernate.generate_statistics=true`, `datasource-proxy`. Решения: `JOIN FETCH`, `@EntityGraph`, `@BatchSize(100)`, `@Fetch(FetchMode.SUBSELECT)`, DTO-проекция. *EAGER* — неправильный ответ.

- Оптимистичная блокировка.

→ `@Version (int/long/Timestamp)`. `UPDATE ... WHERE version = ?`. Если не совпала — *OptimisticLockException*. Подходит для *low-contention* (веб-приложения). Пессимистичная: `@Lock(PESSIMISTIC_WRITE)` → `SELECT FOR UPDATE` — блокирует строку.

- Кэши *Hibernate*.

→ *First-level*: автоматический, привязан к *Session*. Гарантирует *identity* (один PK = один объект). *Second-level*: опциональный, общий (*EhCache*, *Caffeine*, *Infinispan*). *Query cache*: кэширует JPQL-результаты. В банковских проектах: *first-level* всегда, *second-level* осторожно.

- Spring Data — как работает?

→ Интерфейс `extends JpaRepository<Entity, ID>`. Spring создаёт прокси в рантайме. Имя метода парсится: `findByEmailAndStatus` → JPQL. `@Query` для сложных запросов. `Pageable` → `Page/Slice`. *JpaRepository* добавляем `flush()`, `saveAllAndFlush()`, `deleteAllInBatch()` к *CrudRepository*.

ЛОВУШКА • Про EAGER

«Просто поставить EAGER» — неправильный ответ на N+1. EAGER грузит коллекцию ВСЕГДА, даже когда она не нужна — медленнее и съедает память. Правильно: LAZY по умолчанию + `FETCH/EntityGraph` там, где нужны данные. Золотое правило: все `@OneToMany` и `@ManyToMany` — LAZY.

08 SQL и PostgreSQL

По отзывам кандидатов Альфа-Банка: SQL спрашивают отдельно — пишешь запросы вживую. Готовь JOIN, GROUP BY, HAVING, оконные функции, EXPLAIN ANALYZE.

- ACID.
 - Atomicity — целиком или никак. Consistency — валидное состояние БД (constraints). Isolation — параллельные транзакции изолированы (зависит от уровня). Durability — после COMMIT данные сохраняются (WAL в PostgreSQL).
- Уровни изоляции.
 - READ_UNCOMMITTED — dirty read. READ_COMMITTED (default PG) — только закоммиченные. REPEATABLE_READ — повторное чтение = тот же результат. SERIALIZABLE — полная изоляция. Выше уровень → больше консистентности, меньше параллелизма, больше шанс deadlock.
- Типы индексов PostgreSQL.
 - B-tree (default) — =, <, >, BETWEEN, ORDER BY, LIKE 'abc%'. Hash — только =. GIN — массивы, JSONB, full-text. GiST — геометрия, диапазоны. BRIN — большие таблицы с порядком (timestamp). Покрывающий индекс (INCLUDE) — добавляет колонки в лист, Index Only Scan.
- Когда индекс НЕ поможет?
 - Маленькие таблицы. Низкая selectivity (boolean). Функции (LOWER(email) — нужен expression index). LIKE '%abc' (wildcard в начале). Часто обновляемые колонки (индекс замедляет INSERT/UPDATE/DELETE).
- Оконные функции.
 - ROW_NUMBER(), RANK(), DENSE_RANK() — нумерация/ранжирование. LAG/LEAD — предыдущая/следующая строка. SUM/AVG/COUNT OVER (PARTITION BY ... ORDER BY ...) — агрегация без GROUP BY. Пример: зарплата сотрудника vs средняя по его отделу.
- MVCC.
 - Каждая строка: xmin (создавшая транзакция), xmax (удалившая/обновившая). Читатели не блокируют писателей. Snapshot isolation. Старые версии чистит VACUUM (autovacuum). Проблема: table bloat если VACUUM не успевает.
- EXPLAIN ANALYZE.
 - EXPLAIN — план без выполнения. EXPLAIN ANALYZE — реальное выполнение + actual time. Seq Scan на большой таблице — плохо. Index Scan / Index Only Scan — хорошо. Rows Removed by Filter — индекс не помогает. estimated ≠ actual rows — нужен ANALYZE.
- WHERE vs HAVING.
 - WHERE — до группировки, работает с индексами. HAVING — после GROUP BY, для агрегатов. Пример: HAVING COUNT(*) > 5. Ошибка: ставить в HAVING то, что можно в WHERE — медленнее.
- DELETE vs TRUNCATE.
 - DELETE — построчно, триггеры, WAL, можно WHERE, можно откатить. TRUNCATE — мгновенная очистка, сброс автоинкремента, не пишет каждую строку в WAL.

09 Микросервисы и Kafka

Микросервисная архитектура — основа стека Альфы. Kafka — главный брокер. Готовься объяснять не только «что это», но и «что делать, когда что-то пошло не так».

- Что такое микросервис?

→ Автономный сервис с одной зоной ответственности, собственной БД, независимым деплоем. Общается через REST/gRPC (синхронно) или брокер (асинхронно). Плюсы: независимый деплой, масштабирование, технологическая гибкость. Минусы: распределённые транзакции, сетевая латентность, сложность отладки.

- Kafka — зачем?

→ Распределённый лог-ориентированный брокер. Данные хранятся на диске (retention), не удаляются после прочтения. Topic → Partition → Offset. Partition — единица параллелизма. Зачем: асинхронное взаимодействие, буферизация нагрузки, event sourcing, аудит-лог.

- Consumer Group.

→ Каждая партиция — ровно один consumer из группы. Больше партиций → больше параллелизма. Если consumers > partitions — лишние простаивают. Rebalancing при добавлении/падении consumer. Offset commit: auto (рискованно) или manual (контролируемо).

- Гарантии доставки.

→ at-most-once — может потеряться. at-least-once — может продублироваться (default). exactly-once — idempotent producer + transactional + read_committed. В проде: at-least-once + идиоматичная обработка (upsert в БД, дедупликация по event_id).

- Circuit Breaker.

→ Защита от каскадных сбоев. CLOSED → OPEN (блокирует) → HALF_OPEN (тестовые запросы). Resilience4j + Spring Boot стартёр. Конфигурация: failureRateThreshold, waitDurationInOpenState. Fallback: дефолтное значение или кэш.

- Saga.

→ Распределённые транзакции через цепочку локальных + компенсирующие действия. Choreography (события, децентрализованно) vs Orchestration (центральный оркестратор). Пример: оплата → списание → бронирование. Бронирование упало → возврат → отмена.

- Idempotent consumer.

→ Таблица processed_events(event_id UUID PK). INSERT перед обработкой — конфликт → skip. Или upsert по бизнес-ключу (ON CONFLICT DO UPDATE). В транзакции с бизнес-логикой.

10 Docker, Kubernetes, CI/CD

- Образ vs контейнер.

→ Образ — неизменяемый шаблон из слоёв (каждая инструкция *Dockerfile* — слой). Контейнер — запущенный экземпляр с *writable*-слоем. *docker images* → образы, *docker ps* → контейнеры.

- Multi-stage build.

→ *FROM maven:3.9* → сборка. *FROM eclipse-temurin:21-jre* → копируем только JAR. Финальный образ ~200 MB вместо ~800 MB. Безопаснее (меньше *attack surface*), быстрее деплой.

- Kubernetes: Pod, Deployment, Service.

→ Pod — 1+ контейнер, общий *network*. Deployment — управляет *ReplicaSet*: реплики, *rolling update*, *rollback*. Service — стабильный IP/DNS: *ClusterIP* (внутренний), *NodePort*, *LoadBalancer*. Ingress — L7 маршрутизация.

- Liveness vs Readiness vs Startup probe.

→ *Liveness* — жив ли? Нет → перезапуск. *Readiness* — готов к трафику? Нет → убирается из LB. *Startup* — для медленного старта: пока не пройдёт, остальные не проверяются. *Spring Boot Actuator*: */actuator/health/liveness*, */actuator/health/readiness*.

- CI vs CD.

→ CI — автосборка+тесты при *push*. CD — *Delivery* (ручной деплой) или *Deployment* (автодеплой). Pipeline: *checkout* → *build* → *test* → *docker build* → *push registry* → *deploy staging* → (*approval*) → *prod*.

11 Тесты: JUnit, Mockito, TDD

В вакансиях Альфы тесты — обязательное требование: JUnit 5, Testcontainers, WireMock. Могут попросить написать тест прямо на собеседе.

- Пирамида тестирования.
 - Unit (много, быстро) → интеграционные (связь компонентов) → e2e (мало, хрупко). В банках: обязательно unit + integration. E2e — на критичных сценариях.
- mock vs spy.
 - mock() — пустой объект, всё default. spy() — обёртка над реальным: методы вызываются, если не замокать. Для spy: doReturn().when(spy).method() (иначе вызовется реальный метод).
- ArgumentCaptor.
 - Захват аргумента: verify(repo).save(captor.capture()); assertThat(captor.getValue().getName()).isEqualTo("Alice"). Когда нужно проверить не факт вызова, а конкретные значения.
- Testcontainers.
 - Реальные БД/Кafka в Docker из тестов. @Testcontainers + @Container PostgreSQLContainer. Зачем: тесты на реальной PostgreSQL, а не H2 (которая отличается). Проверка миграций Flyway, SQL, Kafka-consumers.
- TDD.
 - Red: падающий тест. Green: минимальный код. Refactor: улучшаем без изменения поведения. Private-методы не тестируем — через публичные. Если сложно — выдели класс.
- WireMock.
 - Мок HTTP-сервисов для интеграционных тестов. WireMockServer или @WireMockTest. stubFor(get("/api/...").willReturn(okJson("..."))). Позволяет тестировать взаимодействие с внешними API без реального вызова.

12 Практические задачи (live-coding)

По отзывам кандидатов: задачи прикладные, без LeetCode Hard. REST-контроллер, unit-тест, code review, Stream API, SQL. Рассуждай вслух!

► Задача 1. Найти дубликаты

Дан `List<Integer>`. Вернуть `Set<Integer>` чисел, встречающихся более одного раза.

```
public Set<Integer> findDuplicates(List<Integer> list) {
    Set<Integer> seen = new HashSet<>();
    Set<Integer> duplicates = new HashSet<>();
    for (Integer n : list) {
        if (!seen.add(n)) {
            duplicates.add(n);
        }
    }
    return duplicates;
}
```

$O(n)$ время, $O(n)$ память. `seen.add()` возвращает `false`, если элемент уже есть — элегантная проверка. Альтернатива через Stream: `Collectors.groupingBy + counting() + filter > 1`.

► Задача 2. Группировка Stream API

```
// Отдел → список сотрудников
Map<String, List<Employee>> byDept = employees.stream()
    .collect(Collectors.groupingBy(Employee::getDepartment));

// Отдел → количество
Map<String, Long> counts = employees.stream()
    .collect(Collectors.groupingBy(
        Employee::getDepartment, Collectors.counting()));

// Отдел → средняя зарплата
Map<String, Double> avgSalary = employees.stream()
    .collect(Collectors.groupingBy(
        Employee::getDepartment,
        Collectors.averagingDouble(Employee::getSalary)));
```

► Задача 3. SQL вживую

Отделы со средней зарплатой > 100 000:

```
SELECT d.name, AVG(e.salary) AS avg_salary
FROM employees e
JOIN departments d ON d.id = e.department_id
GROUP BY d.name
HAVING AVG(e.salary) > 100000
ORDER BY avg_salary DESC;
```

Индексы: `employees.department_id`, покрывающий (`department_id, salary`). Продолжат: сотрудники с зарплатой выше средней по СВОЕМУ отделу → `AVG() OVER (PARTITION BY department_id)`.

► Задача 4. SQL из собеседов Альфы

Покупатели, купившие Laptop И Monitor в марте 2024:

```
SELECT c.name
FROM customer c
JOIN purchase p ON c.customer_key = p.customer_key
JOIN product pr ON p.product_key = pr.product_key
WHERE pr.name IN ('Laptop', 'Monitor')
  AND EXTRACT(MONTH FROM p.date) = 3
  AND EXTRACT(YEAR FROM p.date) = 2024
GROUP BY c.customer_key, c.name
HAVING COUNT(DISTINCT pr.name) = 2;
```

Ключ: HAVING COUNT(DISTINCT pr.name) = 2 — покупатель купил ОБА товара. Без DISTINCT — дубли покупок сломают подсчёт.

► Задача 5. REST-контроллер

```
@RestController
@RequestMapping("/api/users")
public class UserController {

    private final UserService userService;

    public UserController(UserService userService) {
        this.userService = userService;
    }

    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    public UserDto create(@RequestBody @Valid CreateUserRequest req) {
        return userService.create(req);
    }

    @GetMapping("/{id}")
    public UserDto getById(@PathVariable Long id) {
        return userService.findById(id);
    }
}

public record CreateUserRequest(
    @NotBlank String name,
    @Email String email,
    @Min(18) int age
) {}
```

Constructor injection, DTO (не Entity), @Valid, 201 Created для POST, record (Java 16+). Обработка MethodArgumentNotValidException через @RestControllerAdvice.

► Задача 6. Unit-тест Mockito + AssertJ

```
@ExtendWith(MockitoExtension.class)
class UserServiceTest {

    @Mock private UserRepository userRepository;
    @Mock private EmailSender emailSender;
    @InjectMocks private UserService userService;

    @Test
    void shouldRegisterUserAndSendEmail() {
        // given
```

```

    User saved = new User(1L, "Alice", "alice@example.com");
    when(userRepository.save(any(User.class))).thenReturn(saved);
    // when
    User result = userService.register("alice@example.com", "Alice");
    // then
    assertThat(result.getId()).isEqualTo(1L);
    ArgumentCaptor<User> captor = ArgumentCaptor.forClass(User.class);
    verify(userRepository).save(captor.capture());
    assertThat(captor.getValue().getName()).isEqualTo("Alice");
    verify(emailSender).sendWelcome("alice@example.com");
}
}

```

► Задача 7. Потокобезопасный кэш

```

public class SimpleCache {
    private final Map<String, String> cache = new ConcurrentHashMap<>();

    public String get(String key, Function<String, String> loader) {
        return cache.computeIfAbsent(key, loader);
    }
}

```

ConcurrentHashMap (не HashMap — бесконечный цикл в Java 7, потеря данных). computeIfAbsent (не containsKey+put — гонка). Ограничение размера: Caffeine/Guava Cache с maxSize и TTL.

► Задача 8. Code Review

```

public class UserCache {
    private static Map<Long, User> cache = new HashMap<>();

    public static User getUser(Long id) {
        if (cache.containsKey(id)) {
            return cache.get(id);
        }
        User user = loadFromDb(id);
        cache.put(id, user);
        return user;
    }
}

```

Проблемы: 1) HashMap не потокобезопасен → ConcurrentHashMap. 2) containsKey+get → computeIfAbsent. 3) Кэш бесконечный → memory leak → нужен TTL/maxSize. 4) static глобальное состояние → Spring-бин. 5) null от loadFromDb не обработан.

► Задача 9. N+1 — найти и починить

@Entity Order с @OneToMany(mappedBy="order") List<OrderItem> items (LAZY). В сервисе: findAll() + цикл по items.size(). Решения: JOIN FETCH в @Query, @EntityGraph(attributePaths="items"), @BatchSize(size=100), DTO-проекция. «Просто поставить EAGER» — неправильный ответ.

13 План подготовки + чек-лист

► За 2–3 недели

- 20–30 задач LeetCode (Easy + Medium): массивы, строки, HashMap, Stream API
- Pet-проект: Spring Boot 3 + PostgreSQL + Kafka + Testcontainers
- Hibernate: N+1, состояния сущности, JOIN FETCH, @EntityGraph, @Version
- SQL: JOIN, GROUP BY, HAVING, оконные функции, EXPLAIN ANALYZE
- 5–10 unit-тестов с Mockito + AssertJ по TDD
- Docker: Dockerfile multi-stage, docker-compose с БД + Kafka

► За неделю

- 2–3 мок-интервью: pramp.com, interviewing.io или друзья
- Прорешать все вопросы из гайда ВСЛУХ — мысли ≠ слова
- 2–3 истории: проблема → что сделал → результат (с цифрами)
- Решить задачи «что выведет» на Integer cache, Stream API, equals
- Перечитать вакансию Альфы, выписать незнакомые слова

► В день собеседа

- Камера, микрофон, интернет — за 30 минут
- IDE наготове (IntelliJ IDEA), терминал с docker-compose
- Рассуждать вслух — молчание хуже «дай подумать»
- Не знаешь — честно: «не сталкивался, но предположу...»
- 2–3 вопроса в конце: проект, команда, code review, стек

ВНИМАНИЕ · Про банковские проекты

Альфа-Банк проводит проверку СБ — стандарт для банков. Удалёнка возможна (полная или гибридная). IT-хабы: Москва, Питер, Екатеринбург. Сезонный коворкинг в Сочи.

► Финальный чек-лист

Блок	Готов, если можешь...
Java Core	equals/hashCode на примере + как нарушение ломает HashSet
Коллекции	HashMap в Java 8+: бакеты, treeify, resize, mutable-ключ
Многопоточность	Потокобезопасный singleton + volatile в DCL + happens-before
Spring	@Transactional под капотом + self-invocation + Propagation
Hibernate	N+1: найти, объяснить, 2+ способа починить

SQL	JOIN+GROUP BY+HAVING + оконная функция + обосновать индекс
Kafka	Consumer Group, гарантии, идемпотентность
Docker/K8s	Dockerfile multi-stage + Pod/Deployment/Service + probes
Тесты	Mockito + ArgumentCaptor + AssertJ (given-when-then)
Live-coding	15 мин чистое решение + сложность + edge cases

Удачи на собеседе!

// git push origin offer

JavaJub